实验1：DFA 模拟实现

1 实验内容（必做）

1.1 设计思路

1.2 实现过程

1.3 测试用 DFA

2 实验结果（必做）

3 实验内容（选做）

3.1 设计思路

3.2 实现过程

4 实验结果（选做）

5 个人总结

1 实验内容（必做）

编写一个 C 程序，模拟 DFA 的运行过程。程序需要从文本文件中读入 DFA 的五元组定义（字符集、状态集、开始状态、接受状态集、状态转换表），对 DFA 进行合法性检查，输出长度不超过 N 的所有规则字符串，并模拟 DFA 对给定字符串的识别过程。

1.1 设计思路

我采用单文件 C 程序实现，用一个结构体 `DFA` 存储五元组的全部信息。状态用整数编号（从 1 开始），字符集用字符数组存储，状态转换表用二维数组 `transitions[state][symbol_index]` 表示。

DFA 定义文件的格式设计如下：第一行为字符集（如 `ab`），第二行为状态总数，第三行为开始状态编号，第四行为接受状态编号，之后每行对应一个状态在各字符下的转移目标。

程序的整体流程为：读取文件 → 打印 DFA 信息 → 合法性检查 → 枚举规则字符串 → 模拟识别。

1.2 实现过程

1. 文件读取与 DFA 构建

   程序逐行读取文本文件，依次解析字符集、状态数、开始状态、接受状态集和转换表。解析过程中对每一行做基本的格式校验，如果某一行缺失或格式不对则直接报错退出。

2. DFA 合法性检查

   按照实验要求，程序对读入的 DFA 进行四项检查：开始状态是否唯一、开始状态是否在状态集中、接受状态集是否为空、接受状态是否都在状态集中。此外还额外检查了转换表中每个转移目标是否合法。

3. 枚举规则字符串

   采用递归深度优先搜索实现。从开始状态出发，逐层尝试字符集中的每个字符进行状态转移，当到达接受状态时输出当前路径上积累的字符串，当深度达到 N 时回溯。这样可以不遗漏地枚举出所有长度不超过 N 的被接受字符串。

4. 模拟识别过程

   对输入字符串逐字符查找其在字符集中的索引，根据转换表进行状态转移，每一步都打印当前状态和转移后的状态。读完整个字符串后，判断最终状态是否为接受状态，输出结论。程序同时支持手动输入和随机生成两种方式获取待判定字符串。

1.3 测试用 DFA

我构造了一个字符集为 `{a, b}`、共 4 个状态的 DFA，其接受的语言为所有以连续偶数个相同字符结尾的串。开始状态为 1，接受状态为 4。

2 实验结果（必做）

```
wuji@taishan:~/compiler-labs/simple-required$ /home/wuji/compiler-labs/simple-required/main
字符集: ab
状态数: 4
开始状态: 1
接受状态集: 4
状态转移表:
状态 1: a->2 b->3
状态 2: a->4 b->3
状态 3: a->2 b->4
状态 4: a->4 b->4
DFA 检查通过。
请输入最大长度 N: 4
长度不超过 4 的规则字符串如下:
aa
aaa
aaaa
aaab
aab
aaba
aabb
abaa
abb
abba
abbb
baa
baaa
baab
babb
bb
bba
bbaa
bbab
bbb
bbba
bbbb
共 22 个。
请选择字符串来源(1.手动输入 2.随机生成): 1
请输入一个字符串(空串请直接输入 #): aabc
识别过程:
初始状态: 1
读入 a: 1 -> 2
读入 a: 2 -> 4
读入 b: 4 -> 4
字符 c 不属于字符集。
```

```
wuji@taishan:~/compiler-labs/simple-required$ /home/wuji/compiler-labs/simple-required/main
字符集: ab
状态数: 4
开始状态: 1
接受状态集: 4
状态转移表:
状态 1: a->2 b->3
状态 2: a->4 b->3
状态 3: a->2 b->4
状态 4: a->4 b->4
DFA 检查通过。
请输入最大长度 N: 2
长度不超过 2 的规则字符串如下:
aa
bb
共 2 个。
请选择字符串来源(1.手动输入 2.随机生成): 2
请输入随机字符串长度: 8
随机生成的字符串: baaabbba
识别过程:
初始状态: 1
读入 b: 1 -> 3
读入 a: 3 -> 2
读入 a: 2 -> 4
读入 a: 4 -> 4
读入 b: 4 -> 4
读入 b: 4 -> 4
读入 b: 4 -> 4
读入 a: 4 -> 4
结束状态: 4
结论: 该字符串属于此 DFA 的语言。
```

程序正确读入了字符集 `{a, b}`、4 个状态、开始状态 1、接受状态 4 的 DFA 定义，并通过了全部合法性检查。第一次运行设定 N=4，枚举出 22 个规则字符串。手动输入 `aabc` 进行识别，DFA 依次经过 1→2→4→4，在读入第四个字符 `c` 时发现其不属于字符集，识别过程终止。第二次运行设定 N=2，枚举出 `aa` 和 `bb` 两个规则字符串。选择随机生成长度为 8 的字符串 `baaabbba`，DFA 从状态 1 出发经过 3→2→4→4→4→4→4→4，结束状态为 4，属于接受状态，判定该字符串属于此 DFA 的语言。

3　实验内容（选做）

选做部分要求结合教材 1.6 节主文法的概念，完成一个面向简化程序设计语言构造的 DFA 编程实现。

3.1　设计思路

教材指出，主文法是一个面向命令式程序设计语言主要语言构造的识别器，目标是抓住核心语法结构并去掉琐碎部分。我选择对四种基本语句进行建模：赋值、条件、循环和输出。

由于 DFA 的字符集必须是有限的，而标识符和整数本身由无限多种具体串组成，我采用了 token 化的思路，将词法单元映射为单个符号：

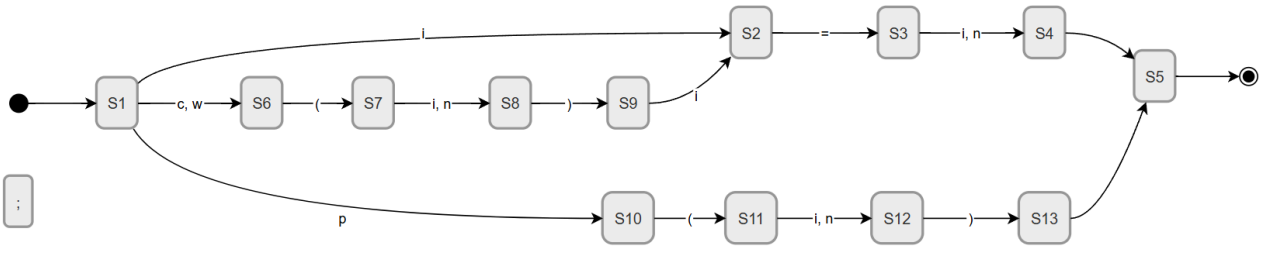
token	含义
`i`	标识符
`n`	整数字面量
`c`	if 关键字
`w`	while 关键字
`p`	print 关键字
`= ; ( )`	原样保留

在这套字符集 `{i, n, =, ;, (, ), c, w, p}` 上，四种语句的 token 模式分别为：

- 赋值：`i=i;` `i=n;`
- 条件：`c(i)i=i;` `c(n)i=n;` 等
- 循环：`w(i)i=i;` `w(n)i=n;` 等
- 输出：`p(i);` `p(n);`

条件和循环语句的结构是关键字后跟括号内的条件表达式，再跟一条赋值语句作为语句体。这里不处理嵌套，因为嵌套结构属于上下文无关语言，超出了 DFA 的能力范围。

基于以上分析，我设计了一个 14 状态的 DFA。状态 1 为起始，状态 5 为唯一的接受状态，状态 14 为死状态。赋值语句走 1→2→3→4→5 的路径，条件和循环语句先经过 6→7→8→9 完成条件部分的识别再汇入赋值路径，输出语句走 10→11→12→13→5 的独立路径。以下是该 DFA 的状态转移图：

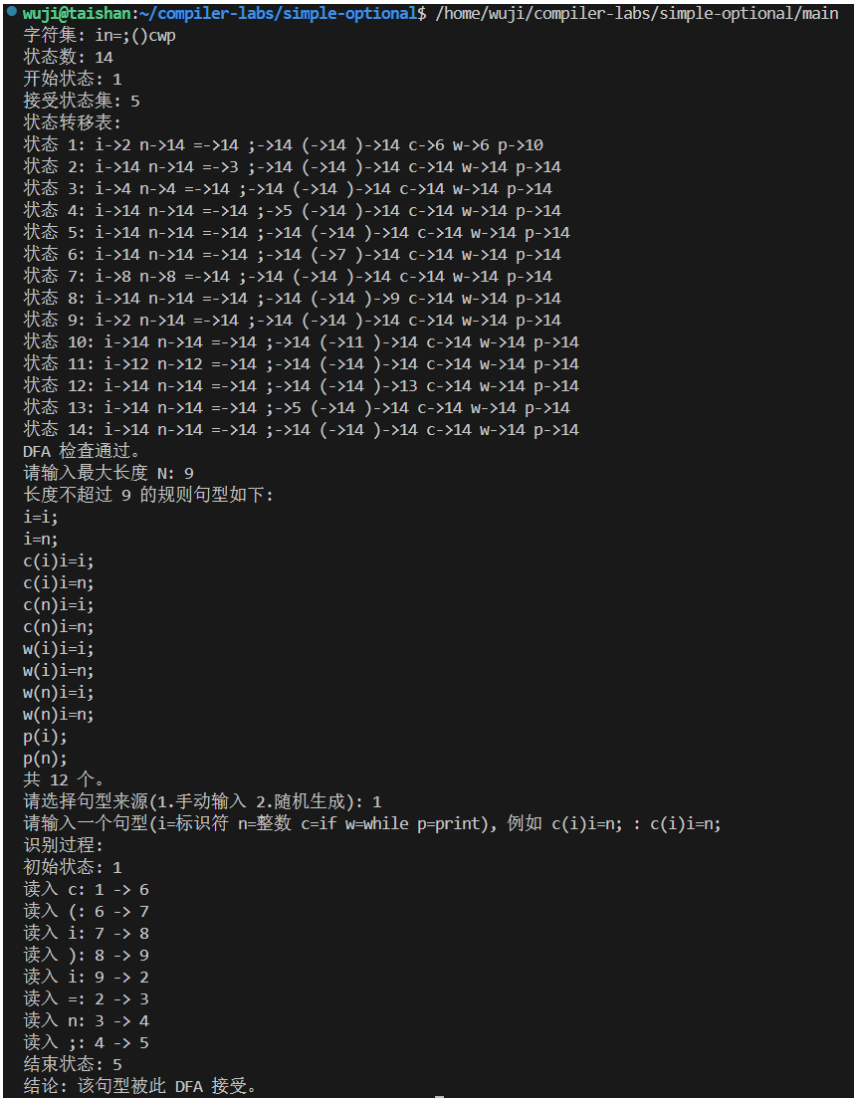


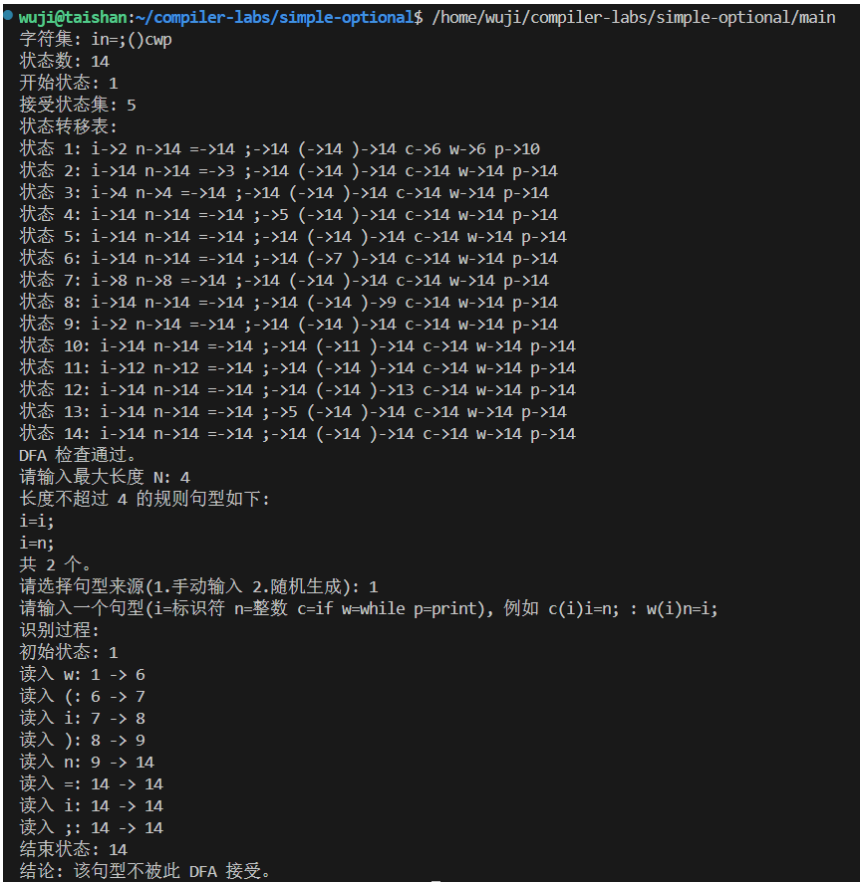
3.2　实现过程

选做部分复用了必做部分的程序框架，同样从文件读入 DFA 定义、进行合法性检查、支持枚举和模拟识别。区别仅在于 DFA 定义文件不同：字符集扩展为 9 个 token，转换表对应 14 状态的设计。

枚举功能可以直接验证 DFA 的正确性：设定 N=9 时，程序输出全部 12 个规则句型，覆盖了四种语句类型的所有合法组合，与设计预期一致。

4 实验结果（选做）


wuji@taishan:~/compiler-labs/simple-optional$ /home/wuji/compiler-labs/simple-optional/main
字符集: in=;()cwp
状态数: 14
开始状态: 1
接受状态集: 5
状态转移表:
状态 1: i->2 n->14 =->14 ;->14 (->14)->14 c->6 w->6 p->10
状态 2: i->14 n->14 =->3 ;->14 (->14)->14 c->14 w->14 p->14
状态 3: i->4 n->4 =->14 ;->14 (->14)->14 c->14 w->14 p->14
状态 4: i->14 n->14 =->14 ;->5 (->14)->14 c->14 w->14 p->14
状态 5: i->14 n->14 =->14 ;->14 (->14)->14 c->14 w->14 p->14
状态 6: i->14 n->14 =->14 ;->14 (->7)->14 c->14 w->14 p->14
状态 7: i->8 n->8 =->14 ;->14 (->14)->14 c->14 w->14 p->14
状态 8: i->14 n->14 =->14 ;->14 (->14)->9 c->14 w->14 p->14
状态 9: i->2 n->14 =->14 ;->14 (->14)->14 c->14 w->14 p->14
状态 10: i->14 n->14 =->14 ;->14 (->11)->14 c->14 w->14 p->14
状态 11: i->12 n->12 =->14 ;->14 (->14)->14 c->14 w->14 p->14
状态 12: i->14 n->14 =->14 ;->14 (->14)->13 c->14 w->14 p->14
状态 13: i->14 n->14 =->14 ;->5 (->14)->14 c->14 w->14 p->14
状态 14: i->14 n->14 =->14 ;->14 (->14)->14 c->14 w->14 p->14
DFA 检查通过。
请输入最大长度 N: 9
长度不超过 9 的规则句型如下:
i=i;
i=n;
c(i)i=i;
c(i)i=n;
c(n)i=i;
c(n)i=n;
w(i)i=i;
w(i)i=n;
w(n)i=i;
w(n)i=n;
p(i);
p(n);
共 12 个。
请选择句型来源(1.手动输入 2.随机生成): 1
请输入一个句型(i=标识符 n=整数 c=if w=while p=print), 例如 c(i)i=n; : c(i)i=n;
识别过程:
初始状态: 1
读入 c: 1 -> 6
读入 (: 6 -> 7
读入 i: 7 -> 8
读入): 8 -> 9
读入 i: 9 -> 2
读入 =: 2 -> 3
读入 n: 3 -> 4
读入 ;: 4 -> 5
结束状态: 5
结论: 该句型被此 DFA 接受。



程序读入了扩展后的 9 符号字符集和 14 状态转换表，验证通过。设定 N=9 后枚举出全部 12 个规则句型，覆盖赋值、条件、循环、输出四种语句类型，与设计预期一致。对合法串 `c(i)i=n;` 的识别过程中，DFA 依次经过 1→6→7→8→9→2→3→4→5，最终到达接受状态 5。对非法串 `w(i)n=i;` 的识别过程中，DFA 在状态 9 读入 `n` 时转入死状态 14，后续字符均停留在死状态，最终被拒绝。这说明 DFA 正确地约束了条件、循环语句体必须以标识符开头的赋值语句。

5　个人总结

本实验围绕 DFA 的定义、验证和模拟识别展开，完成了从文件读入五元组、检查合法性、枚举规则字符串到逐步模拟识别的完整流程。在实现过程中，我对 DFA 的工作机制有了更具体的认识。DFA 的核心就是一张确定的状态转换表，给定当前状态和输入符号，下一步转移到哪个状态是唯一确定的。枚举规则字符串的过程本质上是对转换表做深度优先遍历，状态有限、字符集有限，所以长度受限的被接受串也一定是有限可枚举的，在实际跑出枚举结果后能够更清晰地认识到这一点。

选做部分让我理解了如何将实际的语言构造抽象到 DFA 能处理的层面。赋值、条件、循环、输出这些语句涉及标识符和整数这类无限集合，但通过 token 化把它们各自压缩为一个有限符号后，问题就回到了有限字符集上的正则语言识别。同时我也体会到 DFA 的局限性，它只能处理扁平的语句模式，一旦涉及嵌套结构就需要更强的识别模型。这也是编译器前端词法分析和语法分析分层处理的基本思路，本次实验算是对这个过程的一次小规模实践。